

Projet MIPP : HelpDesk AI : gestion de tickets support et priorisation intelligente

Table des matières

1	Introduction	2
2	Expression et synthèses des besoins	3
2.1	Les acteurs	3
2.2	Règles de gestion	3
2.2.1	Création de ticket	3
2.2.2	Cycle de vie d'un ticket	3
2.2.3	Assignation	3
2.2.4	Commentaires et historique	4
2.2.5	Persistence	4
2.2.6	Cas limites et validations	4
3	Modélisation UML du problème	5
3.1	Diagramme des cas d'utilisation	5
3.2	Diagramme des classes	5
4	Implémentation JAVA du système	6
4.1	Architecture générale	6
4.2	Gestion des droits par héritage	6
4.3	Traçabilité des actions	6
4.4	Validation des saisies	6
4.5	Intégration de l'IA	7
4.6	Sauvegarde	7
5	Conclusion	8
6	Bibliographie	9

1 Introduction

Dans les organisations modernes, qu'il s'agisse de grandes entreprises, de PME, d'administrations publiques ou d'établissements d'enseignement, les services informatiques sont devenus le pilier central du bon fonctionnement des activités. Les collaborateurs dépendent quotidiennement d'outils numériques : messageries, applications métier, accès réseau, postes de travail. Toute interruption, même brève, peut engendrer des pertes de productivité significatives. Face à cette dépendance croissante, le volume des demandes d'assistance adressées aux équipes IT ne cesse d'augmenter. Pannes matérielles, comptes bloqués, problèmes de connectivité, installations logicielles, questions de sécurité : autant d'incidents qui arrivent simultanément, sans ordre de priorité apparent, et qui doivent être traités dans des délais raisonnables. Face à ce flux continu de demandes, la gestion par e-mail ou par téléphone devient rapidement ingérable. Un outil de ticketing aussi appelé HelpDesk apporte une réponse structurée à ce problème en centralisant, traçant et priorisant chaque demande sous la forme d'un ticket numéroté. On se propose alors dans ce projet d'implémenter en langage JAVA, un système de gestion et de priorisation des tickets, incluant une aide de l'IA pour déterminer la priorité, et proposer une piste de solution du ticket.

2 Expression et synthèses des besoins

2.1 Les acteurs

Le système met en jeu trois catégories d'acteurs.

Le demandeur est tout collaborateur de l'organisation qui rencontre un problème informatique. Il peut créer un ticket, consulter uniquement ses propres tickets et ajouter des commentaires pour apporter des précisions.

Le technicien est un agent du support informatique. Il accède à l'ensemble des tickets, peut s'auto-assigner un ticket non encore pris en charge, modifier le statut, la catégorie ou la priorité, laisser des commentaires et solliciter le module IA pour une suggestion de résolution.

Le responsable hérite de toutes les capacités du technicien et dispose en plus du droit de d'assigner un ticket déjà attribué ou non à un technicien. Il est le seul acteur autorisé à effectuer cette opération.

2.2 Règles de gestion

2.2.1 Création de ticket

Un ticket est créé exclusivement par un demandeur. Les champs titre, description et catégorie sont obligatoires et soumis à des contraintes de longueur. À la création, le module IA analyse la description pour valider ou ajuster la priorité indiquée. En cas d'indisponibilité de l'IA, la priorité saisie est conservée sans erreur bloquante. Chaque ticket reçoit un identifiant entier unique auto-incrémenté et un horodatage automatique.

2.2.2 Cycle de vie d'un ticket

Un ticket suit le cycle : `OUVERT` → `EN_COURS` → `RESOLU` → `CLOS`. Seuls les techniciens et responsables peuvent faire évoluer le statut. Le statut d'un ticket peut retourner à un état précédent si nécessaire. Chaque changement est tracé dans l'historique avec la date et l'heure.

2.2.3 Assignment

Un ticket non assigné (valeur "`aucun`") peut être pris en charge par n'importe quel technicien ou responsable. Un ticket déjà assigné ne peut être réassigné que par un responsable. Toute tentative non autorisée est rejetée avec un message explicite.

2.2.4 Commentaires et historique

Demandeurs et techniciens peuvent ajouter des commentaires, horodatés et nominatifs. Chaque action significative (création, changement de statut, de priorité, de catégorie, assignation, commentaire) est consignée automatiquement dans l'historique interne du ticket.

2.2.5 Persistance

L'état complet du système est sauvegardé dans un fichier `data/tickets.csv` après chaque modification et rechargé au démarrage, garantissant la continuité entre les sessions.

2.2.6 Cas limites et validations

Les champs texte ne peuvent contenir les séquences réservées `|||`, `~~`, `##` et `[NEWLINE]`, utilisées comme délimiteurs CSV. Toute saisie invalide (énumération inconnue, entier non positif, champ vide) provoque un message d'erreur et une nouvelle demande de saisie, sans interruption du programme.

3 Modélisation UML du problème

3.1 Diagramme des cas d'utilisation

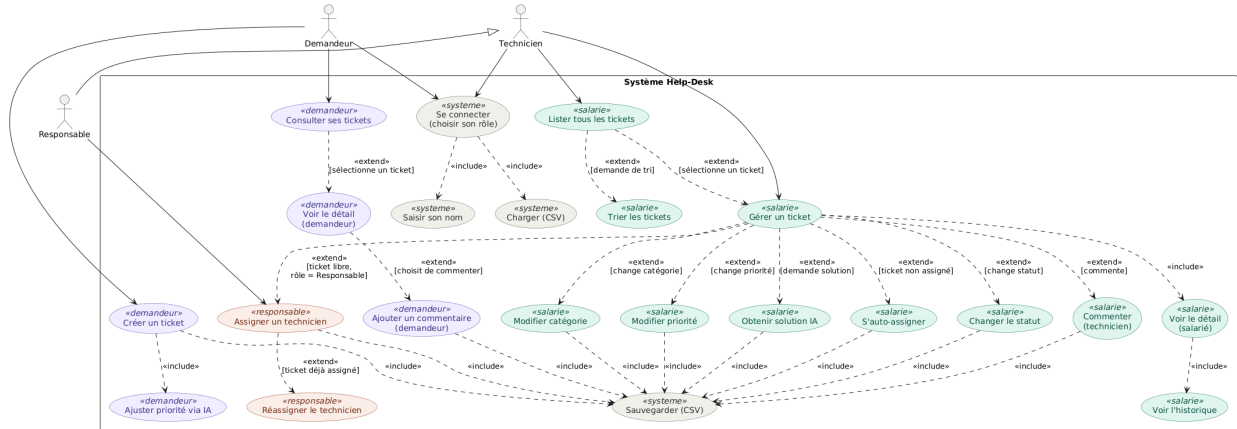


FIGURE 3.1 – Diagramme des cas d'utilisation du système HelpDesk

3.2 Diagramme des classes

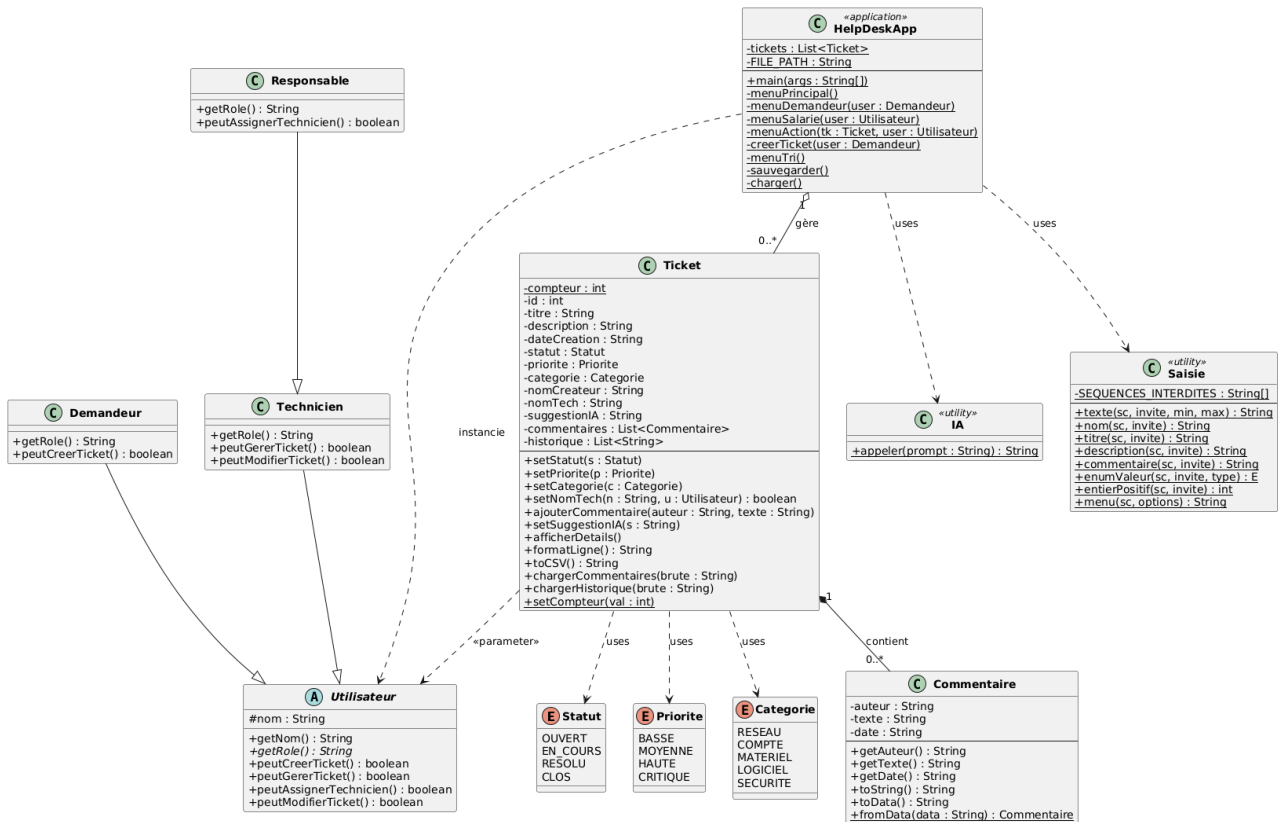


FIGURE 3.2 – Diagramme de classes du système HelpDesk

4 Implémentation JAVA du système

4.1 Architecture générale

Le projet est structuré autour de quatre couches logiques :

Couche	Classes	Rôle
Application	HelpDeskApp	Menus, orchestration, sauvegarde
Modèle	Ticket, Commentaire	Données et logique métier
Utilisateurs	Utilisateur, Demandeur, Technicien, Responsable	Rôles et permissions
Services	IA, Saisie	Validation des entrées, appel IA
Énumérations	Statut, Priorite, Categorie	Domaines de valeurs

4.2 Gestion des droits par héritage

Le choix de centraliser les permissions dans la classe abstraite `Utilisateur` avec des valeurs par défaut à `false` permet de définir clairement les rôles de chaque acteur. L'ajout d'un nouveau rôle se fait uniquement en créant une nouvelle sous-classe, sans modifier le code existant.

4.3 Traçabilité des actions

Chaque setter de `Ticket` enregistre l'ancienne et la nouvelle valeur avant d'effectuer la modification :

```
public void setPriorite(Priorite p) {
    ajouterEvenement("Priorite : " + this.priorite + " -> " + p);
    this.priorite = p;
}
```

Listing 4.1 – Setter traceur dans Ticket.java

L'historique complet est affiché dans la vue détail du ticket et sauvegardé en CSV, garantissant une traçabilité totale sans code supplémentaire dans la couche application.

4.4 Validation des saisies

La classe `Saisie` centralise toutes les entrées utilisateur avec des boucles de retry qui redemandent la saisie en cas d'erreur, sans jamais lever d'exception vers l'appelant. Les contraintes appliquées sont : non-vide, longueur minimale et maximale selon le champ, caractères autorisés pour les noms, absence des séquences réservées CSV, valeur présente dans l'énumération cible, et entier strictement positif.

4.5 Intégration de l'IA

Le module IA s'appuie sur Ollama exécuté localement avec le modèle Mistral. Deux usages sont prévus. A la création du ticket, l'IA reçoit la description et répond par un unique mot parmi **BASSE**, **MOYENNE**, **HAUTE** ou **CRITIQUE**. Si la priorité suggérée diffère de celle du demandeur, elle est automatiquement ajustée avec un message explicatif. Si la réponse est invalide ou si le réseau est indisponible, la priorité initiale est conservée sans erreur bloquante.

```
String repIA = IA.appeler(prompt);
try {
    Priorite pIA = Priorite.valueOf(repIA.toUpperCase().trim());
    if (pIA != pInitial) {
        tk.setPriorite(pIA);
    }
} catch (Exception e) {
    System.out.println("IA indisponible, priorite saisie conservee.");
}
```

Listing 4.2 – Appel IA et fallback dans HelpDeskApp.java

Sur demande pour un ticket existant, le technicien peut demander une suggestion de résolution. La réponse est stockée dans le ticket et sauvegardée en CSV.

4.6 Sauvegarde

La liste des tickets est sauvegardée dans un fichier CSV. Le format de sérialisation utilise `|||` comme séparateur principal entre champs, `~~` entre commentaires, et `##` entre entrées d'historique. Les sauts de ligne dans les réponses IA sont encodés `[NEWLINE]`. Ces séquences étant interdites à la saisie, il n'y a pas de risque de collision. Le chargement reconstruit l'intégralité de l'état en mémoire, y compris l'historique et les commentaires avec leurs horodatages d'origine.

5 Conclusion

Ce projet a permis de concevoir et de réaliser une application de gestion de tickets (HelpDesk) fonctionnelle en Java. L'objectif était de passer d'une gestion de support informelle (e-mails, appels) à un système centralisé et organisé. Nous avons pour cela mis en place une architecture séparant les données, les utilisateurs et la logique de l'application. Le système gère trois profils (demandeur, technicien, responsable) avec des droits spécifiques pour chacun. Un suivi des tickets : Chaque demande suit un cycle de vie précis, de son ouverture à sa clôture. Toutes les actions (changements de statut, commentaires) sont enregistrées automatiquement pour garder une trace de ce qui a été fait. De nombreuses pistes d'améliorations sont également possible. Nous pouvons notamment amélioré le fallback de l'aide IA car actuellement, si l'IA est indisponible, le système conserve la priorité saisie par l'utilisateur. Une amélioration intéressante serait d'implémenter un système de secours basé sur des mots-clés. En analysant la présence de termes critiques (ex : "panne totale", "serveur hors service", "urgent") dans la description, le programme pourrait ajuster la priorité de manière autonome, même sans connexion au module IA. L'interface graphique peut également être améliorée pour une meilleure prise en main du système.

6 Bibliographie

OSMANI, AOMAR. COURS DE CONCEPTION ET PROGRAMMATION OBJET. INSA ROUEN NORMANDIE

DOCUMENTATION JAVA <https://docs.oracle.com/javase/8/docs/api/>